NOTA SOBRE ESTA VERSÃO

Este é o arquivo `artigo_revisado.tex` – cópia do `Artigo.tex` da branch `main` (versão mais atual), com uma camada adicional de revisão: o parecer de um professor/revisor externo, em caixas **vermelhas** (problema grave, resolver antes da banca), **laranjas** (ajuste necessário, não bloqueante), **azuis** (ponto já bom, com sugestão de refinamento) e **violetas** (pergunta que a banca provavelmente vai fazer). O trecho exato a que cada parecer se refere aparece grifado em amarelo logo antes do comentário. O veredito final e a tabela de parâmetros recomendados estão na última seção, antes das referências.

Trabalho de Conclusão de Curso de Engenharia da Computação

RADARTORRES:
PROTÓTIPO PARA DETECÇÃO, LOCALIZAÇÃO E SELEÇÃO AUTOMÁTICA DE TORRE COM CONTROLADOR HEURÍSTICO-NEUROEVOLUTIVO

RadarTorres:
a prototype for target detection, localization and automatic tower selection with a heuristic-neuroevolutionary controller

Gabriel Vasconcellos de Morais[1]
Matheus Emanoel Gomes Souza[2]
Anker Douglas Loss[3]

RESUMO

A construção de protótipos de sensoriamento com hardware acessível é um recurso didático relevante em Engenharia da Computação, por permitir explorar de forma prática conceitos de aquisição de dados, rastreamento e tomada de decisão automática. Este trabalho apresenta o RadarTorres, um protótipo que detecta um alvo em movimento, estima sua posição e decide, entre um conjunto de torres candidatas posicionadas ao redor de uma base central, qual delas deve responder a esse alvo – problema central que o trabalho busca resolver: como identificar, localizar e atribuir automaticamente a torre mais adequada a cada alvo, de forma auditável e sem depender de sensores ou atuadores de uso industrial. O sistema combina dois sensores ultrassônicos (que medem a distância pelo eco do ultrassom) com quatro câmeras de 720p, processa quadros de vídeo (*frames*) e classifica o alvo por sua cor, formato e tamanho. O protótipo converte as leituras de ângulo e distância em posição cartesiana, exibindo-as em tempo real em um radar circular, seleciona automaticamente a torre mais adequada para cada alvo e registra em auditoria toda ação executada. Essa seleção combina uma heurística determinística, escrita como regras explícitas de engenharia, com uma rede neural que aprende, por meio de um algoritmo genético próprio, a refinar essa escolha e o instante do acionamento. O treinamento dessa rede ocorre inteiramente antes da operação do sistema, em um ambiente simulado que executa gerações de episódios de teste até evoluir os parâmetros da rede; concluído o treinamento, esses parâmetros são incorporados de forma fixa à aplicação, de modo que, durante a execução real, o sistema apenas aplica o que foi aprendido previamente, sem ajustar seus parâmetros em tempo real. O processamento e a tomada de decisão são concentrados em uma aplicação desktop, com arquitetura pensada para permitir evolução

[1]Graduando(a) em Engenharia da Computação pela Faculdade do Centro Leste – UCL. E-*mail*: gabrielmorais@ucl.br
[2]Graduando(a) em Engenharia da Computação pela Faculdade do Centro Leste – UCL. E-*mail*: matheusemanoel@ucl.br
[3]Orientador(a). Mestre em xxx pela xxx. E-*mail*: anker@ucl.br

futura sem exigir hardware adicional. Discutem-se, por fim, as limitações inerentes à abordagem de hardware acessível e de baixo custo adotada, e o fato de a validação do protótipo, até o momento, ter sido realizada inteiramente por simulação de software, já que a estrutura física ainda não foi montada nem testada.

Palavras-chave: Detecção e rastreamento de alvos; Seleção automática de torre; Controlador heurístico-neuroevolutivo; Arduino; Protótipo.

ABSTRACT

This work presents RadarTorres, a prototype developed as a Final Course Project in Computer Engineering that detects a moving target, estimates its position, and decides, among a set of candidate towers placed around a central base, which one should respond to that target – the central problem this work addresses: how to automatically identify, localize, and assign the most suitable tower to each target, in an auditable way and without relying on industrial-grade sensors or actuators. The system combines two ultrasonic sensors (which measure distance by ultrasound echo) with four 720p cameras, processes video frames, and classifies the target by color, shape, and size. The prototype converts angle and distance readings into Cartesian position, displaying them in real time on a circular radar, automatically selects the most suitable tower for each target, and logs every executed action for auditing. This selection combines a deterministic heuristic, written as explicit engineering rules, with a neural network that learns, through a custom genetic algorithm, to refine that choice and the activation timing. Training of this network happens entirely before the system is put into operation, in a simulated environment that runs generations of test episodes until the network's parameters evolve; once training is complete, these parameters are embedded as fixed values into the application, so that, during actual execution, the system only applies what was previously learned, without adjusting its parameters in real time. Processing and decision-making are concentrated in a desktop application, with an architecture designed to allow future evolution without requiring additional hardware. Finally, the limitations inherent to the accessible, low-cost hardware approach adopted are discussed, as is the fact that validation of the prototype has so far been carried out entirely through software simulation, since the physical structure has not yet been assembled or tested.

Keywords: Target detection and tracking; Automatic tower selection; Heuristic-neuroevolutionary controller; Arduino; Prototype.

PARECER DO PROFESSOR – BOM, MAS PODE MELHORAR: Este resumo/abstract já cobrem o escopo real do trabalho (controlador heurístico-neuroevolutivo, visão, modelo de dados) – diferente de uma versão anterior que falava só em "modelo de dados". Ponto resolvido, bom trabalho.

1 Introdução

Sistemas automáticos de detecção e resposta a alvos exigem, além de identificar um estímulo, rastrear com precisão sua posição e aplicar regras de decisão que evitem respostas indevidas – tarefas comumente resolvidas com sensores de radar, de custo elevado. Como alternativa a esse tipo de equipamento, este trabalho adota técnicas de visão computacional, aliadas a hardware acessível, como parte central da arquitetura do protótipo. Protótipos didáticos construídos dessa forma desempenham papel importante nesse processo, por permitirem que esses conceitos sejam explorados de forma prática e replicável em laboratório, reproduzindo os princípios de sensoriamento, rastreamento e decisão presentes em sistemas mais sofisticados.

O protótipo resultante, denominado RadarTorres, combina sensores ultrassônicos e câmeras para detectar um alvo, estimar sua posição em um radar circular e decidir, entre um conjunto de torres candidatas dispostas ao redor de uma base central, qual delas deve responder a esse alvo. Essa decisão não é tomada apenas por regras fixas: uma rede neural, evoluída por um algoritmo genético próprio, aprende a refinar a escolha da torre e o instante do acionamento, sempre dentro dos limites que uma camada de regras explícitas (a heurística) já considera seguros. Esse aprendizado acontece em duas etapas bem separadas: antes da operação do sistema, em um ambiente de treinamento simulado, onde a rede evolui ao longo de muitas gerações de episódios de teste; e, depois de treinada, apenas na forma de inferência durante a execução real, sem qualquer ajuste adicional de seus parâmetros em tempo de operação. O problema central que este trabalho busca resolver é, portanto, como automatizar essa cadeia – detectar, localizar e selecionar a torre mais adequada a cada alvo – de forma auditável e replicável em laboratório, sem depender de sensores ou atuadores de uso industrial.

PARECER DO PROFESSOR – PROBLEMA GRAVE: Essa frase é a única coisa no artigo que funciona como "objetivo", e ela é um objetivo GERAL bem redigido – mas **não existe, em lugar nenhum do texto, uma lista de objetivos específicos**. Isso é uma falha estrutural para um TCC: a banca avalia o trabalho CONTRA objetivos declarados, e sem objetivos específicos explícitos e falseáveis (dá para dizer objetivamente se foram cumpridos ou não), a avaliação vira impressão geral – o que tende a jogar contra vocês, não a favor, dado que o trabalho tem bastante conteúdo técnico sólido para mostrar. **Exijo** uma subseção "Objetivos" logo após este parágrafo, com: (1) este objetivo geral, mantido como está (está bom); (2) de 4 a 6 objetivos específicos, cada um verificável, por exemplo: "implementar e documentar o protocolo de comunicação serial Arduino-aplicação", "implementar seleção automática de torre por heurística em camadas, validada em simulação", "implementar e treinar um controlador neuroevolutivo, com evidência quantitativa de evolução do fitness" (que hoje vocês já têm, na Tabela 1 – é só precisar transformar em objetivo nomeado), "migrar a persistência para SQLite com chaves estrangeiras reais" (também já cumprido – precisa só ser nomeado como tal). Cada um desses tem que reaparecer na Conclusão (que já existe e é boa) com um veredito explícito de cumprido/parcial/não cumprido – hoje a Conclusão sintetiza bem os resultados, mas não amarra isso a nenhum objetivo nomeado, porque nenhum foi declarado antes.

2 Desenvolvimento

2.1 Revisão da literatura e fundamentação teórica

Sistemas de detecção e rastreamento de alvos costumam recorrer a sensores como radar, de custo elevado. Como alternativa, sensores acessíveis podem ser combinados de modo que as limitações de um sejam compensadas pelas qualidades do outro. Sensores ultrassônicos, como o HC-SR04, medem distância com boa confiabilidade em curto alcance, mas não distinguem o tipo do obstáculo; câmeras, por sua vez, fornecem informação semântica, porém dependem das condições de iluminação e exigem maior poder de processamento (MAJONI et al., 2025). A fusão de sensores (*sensor fusion*) combina as duas modalidades para aproveitar essa complementaridade.

Majoni et al. (2025) descrevem uma arquitetura em que um Arduino Mega trata o sensoriamento ultrassônico de baixa latência, enquanto um Raspberry Pi processa as imagens da câmera com uma rede YOLOv4-tiny, com comunicação serial entre os dois. Essa divisão de papéis é análoga à do RadarTorres, em que o Arduino realiza a aquisição e a aplicação desktop concentra o processamento.

A literatura também mostra que sensores ultrassônicos acessíveis podem complementar outras fontes de sensoriamento em sistemas de rastreamento, com a precisão melhorada por predição baseada em filtro de Kalman e fusão com a distância medida pelos ultrassônicos (BAEK et al., 2020). O mesmo trabalho observa que as leituras ultrassônicas se tornam ruidosas em distâncias maiores, o que motiva o uso de filtragem.

Definidos o alvo e sua posição, resta decidir qual recurso deve responder. Na literatura de defesa, esse problema é tratado como avaliação de ameaças e atribuição de armas (*Threat Evaluation and Weapon Assignment*, TEWA), no qual as informações dos sensores alimentam as etapas de detecção, rastreamento e identificação, seguidas da avaliação da ameaça e da atribuição dos recursos. As abordagens desse tipo de problema costumam ser classificadas em ameaça por ameaça, em que os recursos são atribuídos sequencialmente, e multiameaça, em que o conjunto de ameaças é considerado simultaneamente.

O RadarTorres adota uma abordagem simples, por proximidade, aplicada alvo a alvo, com propósito exclusivamente demonstrativo (laser de baixa potência, LED ou simulação, sem armamento real); a abordagem multiameaça fica como trabalho futuro.

O refinamento dessa escolha por proximidade – e o instante de disparo – é feito por uma pequena rede neural *feedforward* (6 entradas, 1 camada oculta de 8 neurônios, 2 saídas, ativação tanh), cujos pesos não são ajustados por retropropagação, mas evoluídos por um algoritmo genético próprio (seleção por torneio, *crossover* uniforme, mutação gaussiana, elitismo), auditável e sem dependência de biblioteca de aprendizado de máquina.

A BANCA VAI PERGUNTAR: Por que 8 neurônios na camada oculta, população 60, taxa de mutação 0,08 e elitismo proporcional a 10 %? O artigo não justifica nenhum desses números com citação nem com experimento próprio – é exatamente a pergunta que uma banca de Engenharia faz: "testaram outros valores?". A resposta documentada hoje é "convenção", o que é aceitável como ponto de partida mas não como justificativa final.

PARECER DO PROFESSOR – AJUSTAR: Recomendação barata: vocês já têm a infraestrutura para isso – o treinamento roda dezenas de milhares de gerações em pouco mais de 1h (ver Tabela 1 mais abaixo, 51 blocos de 500 gerações entre 13h40 e 14h57). Rodem o MESMO treinamento variando só um hiperparâmetro por vez (camada oculta 4/8/16, ou população 30/60/120) e reportem o fitness final de cada combinação numa tabelinha de 3-4 linhas. Isso transforma "escolhemos 8 por convenção" em "testamos 4/8/16 e 8 foi o melhor custo-benefício", frase muito mais defensável – e o custo é de minutos, não de dias, dado o tempo de treino já demonstrado no próprio artigo.

Such et al. (2017) mostram que algoritmos genéticos são uma alternativa competitiva ao treino por gradiente em tarefas de aprendizado por reforço, em redes de porte bem maior do que a aqui empregada (até quatro milhões de parâmetros, contra os 74 parâmetros da rede do RadarTorres). Esse resultado sustenta a escolha do algoritmo genético como método de treinamento, mas não implica, por si só, que aumentar o número de camadas ou neurônios seja desnecessário; por isso, a estratégia adotada para ampliar o desempenho do controlador prioriza o ajuste dos hiperparâmetros da busca evolutiva (tamanho de população, paralelização da avaliação de fitness, enriquecimento do sinal de recompensa), sem descartar, a priori, uma rede maior caso evidências futuras mostrem que a capacidade atual seja o fator limitante.

A arquitetura híbrida do controlador – heurística que restringe o espaço de ações antes da rede decidir, e volta a intervir depois, vetando o disparo – tem paralelo direto com duas técnicas da literatura de aprendizado por reforço. A primeira é o mascaramento de ações inválidas (*action masking*): Huang e Ontañón (2020) mostram, para algoritmos de gradiente de política, que mascarar o espaço de ações antes da amostragem corresponde a uma atualização de gradiente válida e se torna mais importante quanto maior a fração de ações inválidas – o que justifica tecnicamente a etapa em que a camada heurística do RadarTorres restringe as torres candidatas (nunca ocupada, nunca fora de alcance) antes de a rede neural escolher entre elas. A segunda é o *shielding*: Alshiekh et al. (2017) propõem um componente reativo (*shield*) que intercepta a ação escolhida pela política aprendida e a substitui quando ela violaria uma propriedade de segurança expressa em lógica temporal – papel equivalente ao do veto heurístico final do RadarTorres sobre o disparo, que nunca é delegado à rede. A atribuição de recursos defensivos a ameaças, por sua vez, é tratada na literatura de TEWA tanto no caso ameaça por ameaça quanto no caso multiameaça (NAEEM et al., 2009b, 2009a); o RadarTorres adota deliberadamente a abordagem mais simples (proximidade, alvo a alvo), com a multiameaça permanecendo como trabalho futuro já apontado anteriormente.

O impacto do treinamento virtual sobre a execução física do controlador é atenuado por uma decisão de projeto central: o mesmo controlador híbrido – heurística e rede neural combinadas – que decide no episódio simulado é o que decide em produção, consultado pelo algoritmo responsável por selecionar a torre a partir das leituras do Arduino real – não há conversão para um formato intermediário (ONNX, TensorFlow Lite) nem reimplementação em outra linguagem ou plataforma, fontes comuns de divergência entre o comportamento treinado e o executado em sistemas de aprendizado por reforço aplicados a robótica. O genoma evoluído é um vetor plano de pesos e vieses, embutido diretamente na aplicação por uma etapa automática de build, que grava o melhor genoma treinado no executável, de onde é carregado por um componente dedicado em tempo de execução; o que muda do ambiente de treino para a operação real é apenas a origem dos dados de entrada (leitura simulada ou leitura do Arduino), nunca a função de decisão em si. Os

parâmetros que definem o episódio simulado – alcance de engajamento e certeza mínima para autorizar disparo – também deixaram de ser constantes fixas no código e passaram a ser lidos das mesmas configurações de alcance de mira e de limiar de certeza usadas pelo sistema real, de modo que recalibrar o alcance da câmera ou o limiar de certeza no painel de Configurações recalibra também o mundo de treino, reduzindo a divergência entre o que foi aprendido e o que será executado. Por fim, a rede neural nunca decide isoladamente: ela apenas pondera a escolha dentro do que a camada heurística determinística já considera válida (nunca uma torre ocupada, nunca fora de alcance), de modo que a arquitetura foi concebida para limitar o pior caso de uma generalização imperfeita do treino virtual para a física – no limite, o controlador regrediria ao comportamento puramente heurístico, nunca a uma decisão arbitrária; trata-se, porém, de uma propriedade de projeto ainda não submetida a validação em hardware físico real.

2.2 Procedimentos metodológicos

O percurso metodológico deste trabalho não seguiu o fluxo tradicional de especificação prévia seguida de implementação: o protótipo RadarTorres já existia, em desenvolvimento incremental, antes do levantamento formal de requisitos. Optou-se, portanto, por catalogar os requisitos funcionais e não funcionais diretamente a partir do que já estava implementado no código-fonte da aplicação desktop e da documentação técnica já produzida ao longo do projeto, em vez de defini-los a priori em um documento de especificação anterior à codificação. O treinamento do controlador por neuroevolução, descrito na Seção 2.1, segue essa mesma lógica incremental: a arquitetura híbrida heurística+rede neural foi definida e ajustada em paralelo ao restante do sistema, não a partir de um plano de treinamento fechado desde o início.

Esse levantamento resultou em 25 requisitos funcionais (RF01–RF25) e 20 requisitos não funcionais (RNF01–RNF20), classificados segundo dois critérios objetivos: a **prioridade**, atribuída conforme a criticidade de cada requisito para a operação central do sistema (comunicação serial, rastreamento, seleção de torre, modos de operação e segurança recebem prioridade Alta; suporte administrativo e ferramental, Média; conveniências de interface, Baixa); e o **status**, que reflete o estado real de implementação constatado no código no momento do levantamento – Implementado, Parcial ou Planejado –, e não a intenção original de projeto. Cada requisito foi, além disso, rastreado até a classe ou método do código-fonte que o realiza, registro mantido em uma matriz de rastreabilidade dedicada, de modo que qualquer afirmação sobre o grau de implementação do protótipo possa ser verificada diretamente no código. Divergências encontradas entre o comportamento observado no código e a nomenclatura ou fluxo originalmente esperado também foram registradas explicitamente, em vez de omitidas ou corrigidas silenciosamente na documentação.

A parte física do RadarTorres (torres, motores, sensores montados em campo) ainda não foi implementada nem validada. Toda a validação funcional relatada neste trabalho foi realizada por simulação de software: a lógica de rastreamento de alvos e de seleção automática de torre foi exercitada de ponta a ponta sem depender de uma porta serial conectada a hardware real, o que impõe um limite claro ao escopo dos resultados relatados – eles atestam a corretude da lógica de decisão implementada em software, mas não substituem uma validação de campo com sensores e torres físicas, deixada como

trabalho futuro.

A mesma ressalva se aplica, com reforço adicional, ao controlador treinado por neuroevolução: o mundo simulado em que os genomas são avaliados é uma aproximação idealizada do ambiente físico – os alvos nascem na borda de um círculo e avançam de forma quase radial, sem o ruído típico de leituras ultrassônicas reais; a certeza de identificação do alvo pela câmera é modelada como uma função sintética que cresce com o tempo de observação, e a probabilidade de errar o disparo é uma função assumida, crescente com a velocidade do alvo – nenhuma delas calibrada com dados de campo, já que o hardware ainda não foi montado. Essa distância entre simulação e realidade física não é apenas teórica: durante este trabalho, identificou-se e corrigiu-se uma falha em que a certeza de identificação do alvo era sorteada uma única vez, no nascimento, e nunca era atualizada – cerca de 70% dos alvos simulados nasciam permanentemente abaixo do limiar mínimo de disparo e jamais podiam ser abatidos, independentemente da qualidade do genoma, mascarando qualquer evolução real do controlador sob a aparência de estagnação. O episódio é citado aqui como evidência concreta de que uma premissa equivocada na simulação pode invalidar silenciosamente o processo de treinamento sem que isso seja percebido pelas métricas agregadas, reforçando a necessidade de validação cruzada entre o comportamento do genoma treinado e o comportamento observado em campo, com hardware físico, antes de se confiar no controlador em um cenário de operação real – validação que permanece como trabalho futuro, nos mesmos termos já apontados para o restante do protótipo.

A Tabela 1 quantifica o achado, a partir do histórico de treinamento persistido localmente (`historico_treinamento_geracoes` e `marcos_fitness_treinamento`, tabelas descritas na Seção 2.5). Antes da correção (commit `a02d46a`), o fitness médio da população permaneceu estagnado em torno de 9 ao longo de 51 blocos de 500 gerações (25.500 gerações treinadas, entre 13h40 e 14h57 do mesmo dia), mesmo com o melhor indivíduo de cada bloco oscilando entre 98 e 145 – sinal de que indivíduos pontualmente sortudos conseguiam boa pontuação, sem que a população como um todo convergisse, exatamente o efeito esperado de 70% dos alvos nascerem abaixo do limiar de disparo. Depois da correção, a taxa de cobertura (fração de alvos que chegam a ser disparáveis) sobe para 100% em todas as cinco amostras de comportamento do melhor indivíduo registradas na mesma tarde, e o fitness médio da população em uma amostra recente de 300 gerações (27/10, gerações 1141–1440) sobe para 134,17, muito mais próximo do fitness do melhor indivíduo (137,35) do que o patamar de 9 observado antes da correção. O genoma embutido no executável (`genoma-controlador.json`) corresponde à geração 5377, com `MelhorFitness` = 136,98.

PARECER DO PROFESSOR – BOM, MAS PODE MELHORAR: Este é o melhor trecho quantitativo do artigo – número real, antes/depois, commit citado. Excelente. **Mas** há uma lacuna de rastreabilidade que a banca vai notar: a amostra "depois" citada cobre as gerações 1141–1440, enquanto o genoma efetivamente embutido no executável é da geração **5377** – quase 4 mil gerações depois, fora da janela reportada. O fitness do genoma embutido (136,98) é consistente com o fitness médio da amostra (134,17), então provavelmente não há problema real, mas o texto não diz isso explicitamente – um leitor atento pergunta "por que vocês reportam uma amostra de 300 gerações que não inclui a geração realmente usada em produção?". Basta uma frase: "o treinamento continuou além da amostra reportada; a Tabela X mostra que o fitness permaneceu estável nesse intervalo, sustentando a escolha da geração 5377 para o genoma embutido" – SE isso for verdade. Se não for (se o fitness variou bastante entre 1440 e 5377), é melhor reportar a amostra que efetivamente inclui a geração 5377.

A BANCA VAI PERGUNTAR: 25.500 gerações entre 13h40 e 14h57 do mesmo dia é pouco mais de 1h17 – aproximadamente 5,5 gerações por segundo, cada uma avaliando 60 genomas × 3 episódios (ver `EpisodiosPorAvaliacao`). Em qual máquina (CPU, núcleos) isso foi medido? Esse número é relevante porque mostra que a ablação de hiperparâmetros que pedi no comentário anterior é praticamente de graça em termos de tempo – mas, sem reportar o hardware, a banca não tem como avaliar se esse desempenho é esperado ou um sinal de que os episódios simulados são triviais demais.

Table 1: Fitness da população antes e depois da correção do bug de certeza estática (commit `a02d46a`), a partir do histórico de treinamento persistido localmente.

Métrica	**Antes (bug)**	**Depois (corrigido)**
Fitness médio da população	≈ 9,11 (min. 8,37, máx. 9,43; 51 blocos de 500 gerações)	134,17 (min. 121,31, máx. 150,60; amostra de 300 gerações)
Melhor fitness por bloco/geração	97,75 a 144,95 (média 119,62)	137,35 em média (122,81 a 155,47)
Taxa de cobertura dos alvos	não medida diretamente – ≈ 70% dos alvos permanentemente abaixo do limiar	100% (5/5 amostras pós-correção)
Genoma embutido no executável	–	geração 5377, fitness 136,98

2.3 Projeto e implementação

2.3.1 Módulo de detecção de alvos

O módulo de detecção utiliza dois sensores ultrassônicos HC-SR04 instalados de costas um para o outro no topo de um mastro, na torre central. O mastro é movido por um motor de passo 28BYJ-48 que oscila 180°, de modo que, em conjunto, os dois sensores cobrem uma varredura de 360°. Cada sensor emite pulsos de 40 kHz e calcula a distância pelo atraso do eco, segundo a relação $d = \frac{v \cdot \Delta t}{2}$, em que v é a velocidade do som no ar (cerca de 343 m/s à temperatura ambiente). A cada leitura, o Arduino envia à aplicação desktop, pela porta serial, o ângulo e a distância em protocolo de texto simples, que a aplicação converte em posição cartesiana e exibe no radar circular de quatro quadrantes.

O protocolo é textual em ASCII, transportado pela porta serial USB a 9600 bauds por padrão, com cada mensagem terminada por `\n` e composta por um tipo em maiúsculas seguido de pares chave-valor separados por ponto e vírgula.

PARECER DO PROFESSOR – PROBLEMA GRAVE: 9600 bauds é o padrão histórico do Arduino, não um valor calculado para a carga do canal – e o firmware físico (`RadarTorres_Firmware.ino`) adiciona, além das mensagens já existentes, `USONIC`, `CAM`, `CALIB`, `COLOR`, `AIM` e todo o protocolo `GENOMA` (um comando POR PESO da rede – 74 comandos sequenciais só para sincronizar o genoma). Uma mensagem `TARGET;ID=1;ANGLE=45;DIST=2.80\n` tem ≈30 bytes; a 9600 bauds (≈960 bytes/s úteis), isso dá um teto teórico de ≈32 mensagens/s **compartilhadas** entre leituras de 2 sensores ultrassônicos, até 4 câmeras reportando `CAM`, comandos `AIM` para até 4 torres e a sincronização do genoma. O artigo nunca calcula nem mede esse teto, apesar de usar "tempo real" no resumo e no abstract. O próprio código já permite 19200 a 115200 bauds – por que o padrão continua em 9600 se o custo de subir é zero?

Uma leitura de alvo chega na forma `TARGET;ID=1;ANGLE=45;DIST=2.80`, em que `ANGLE` é dado em graus no intervalo $[0, 360)$ e `DIST` em metros. O ângulo segue a convenção de bússola – 0° apontando para o Norte, sentido horário –, escolhida por ser a leitura mais intuitiva para quem opera um radar, e a conversão para coordenadas cartesianas de mundo, feita por uma rotina dedicada de conversão de coordenadas polares para cartesianas, é dada por $x = d \cdot \sin\theta$ e $y = d \cdot \cos\theta$, com θ em radianos. A partir do par (x, y), o quadrante do alvo é determinado pelos sinais das duas coordenadas (por um algoritmo auxiliar de identificação de quadrante) e a distância até cada torre candidata pela distância euclidiana clássica (calculada por uma rotina dedicada de cálculo de distância). Leituras com campos ausentes, não numéricos, com ângulo fora do intervalo válido ou com distância negativa são convertidas em mensagens de erro registradas no console de eventos, nunca em exceções – falhas de sensor ou de cabo produzem degradação graciosa, e não interrupção da aplicação.

2.3.2 Projeto elétrico e estrutural

O projeto elétrico é baseado em um Arduino Uno, plataforma disponível no simulador Tinkercad usado para simular o circuito. Os sinais dos motores de passo são gerados por registradores de deslocamento 74HC595 em cascata, e os motores 28BYJ-48 são acionados por drivers ULN2003, alimentados por uma fonte de 5 V/15 A. Cada uma das quatro torres armadas possui dois motores 28BYJ-48 (movimentos de pan e tilt), um laser de baixa potência e um LED de cor própria; o acionamento é exclusivamente demonstrativo.

PARECER DO PROFESSOR – PROBLEMA GRAVE: Esta descrição está desatualizada em relação ao firmware físico que já existe no mesmo repositório (`Arduino/RadarTorres_Firmware/RadarTorres_Firmware.ino`) – e o próprio arquivo admite isso, no próprio cabeçalho: *"Placa: Arduino MEGA 2560 (substituiu o Uno documentado antes – os 13 motores de passo não cabem nos ∼20 pinos do Uno)"*. O firmware real usa **8 motores NEMA17 + driver A4988/DRV8825** (pan+tilt de 4 câmeras) e **4 motores NEMA17 + A4988/DRV8825** (pan das 4 torres) – nenhum 74HC595, nenhum ULN2003 nas torres armadas (o ULN2003 sobrou só para o motor de varredura central, 28BYJ-48). Ou seja: o texto descreve um projeto elétrico (Uno + 74HC595 + ULN2003 em todas as torres) que o próprio repositório já documenta como **substituído**. **Decisão obrigatória**: esta subseção precisa ser reescrita a partir do firmware real – placa, pinagem, os 8+4 motores NEMA17, e por que o 74HC595 foi abandonado – antes de qualquer apresentação à banca, porque uma pergunta sobre "quantos motores tem cada torre" hoje teria duas respostas diferentes dependendo de qual arquivo do próprio repositório se consulta.

PARECER DO PROFESSOR – AJUSTAR: Adicionalmente: os comentários do próprio firmware admitem que valores como `SWEEP_STEPS_POR_VOLTA = 2048` ("valor típico de referência – CONFIRMAR no motor real antes de usar em produção") e os micropassos do driver das câmeras ("CONFIRMAR configuração física") são **suposições de catálogo, não medições**. Isso é coerente com "o hardware ainda não foi montado", já admitido em outras partes do artigo para o controlador neuroevolutivo – mas essa mesma honestidade precisa aparecer também na seção de projeto elétrico, frase a frase, não só implicitamente nos comentários do código-fonte que a banca não vai ler.

Na torre central, o mastro dos sensores não é acoplado diretamente ao eixo do motor: ele assenta em um disco que gira sobre um rolamento axial 51108, acionado pelo motor por um cubo duplo-D. A passagem dos fios fica próxima ao eixo central, com folga de cabo em vez de anel coletor (*slip ring*), já que o giro é limitado a 180°. A torre central é cerca de 5 cm mais alta que as torres armadas (cerca de 137,5 mm de altura). As peças estruturais são impressas em 3D, em formato de torre, com suporte para rolamento axial na base – o que permite a rotação da estrutura –, comum às cinco torres. As torres armadas têm, além disso, suportes próprios para os motores e para as câmeras, que cumprem o papel dos "canhões" de acionamento demonstrativo; a torre central tem suporte próprio para os sensores ultrassônicos no topo do mastro. Todas essas peças, assim como o restante do projeto, existem até o momento apenas em projeto digital (modelagem 3D e simulação elétrica), sem montagem física realizada.

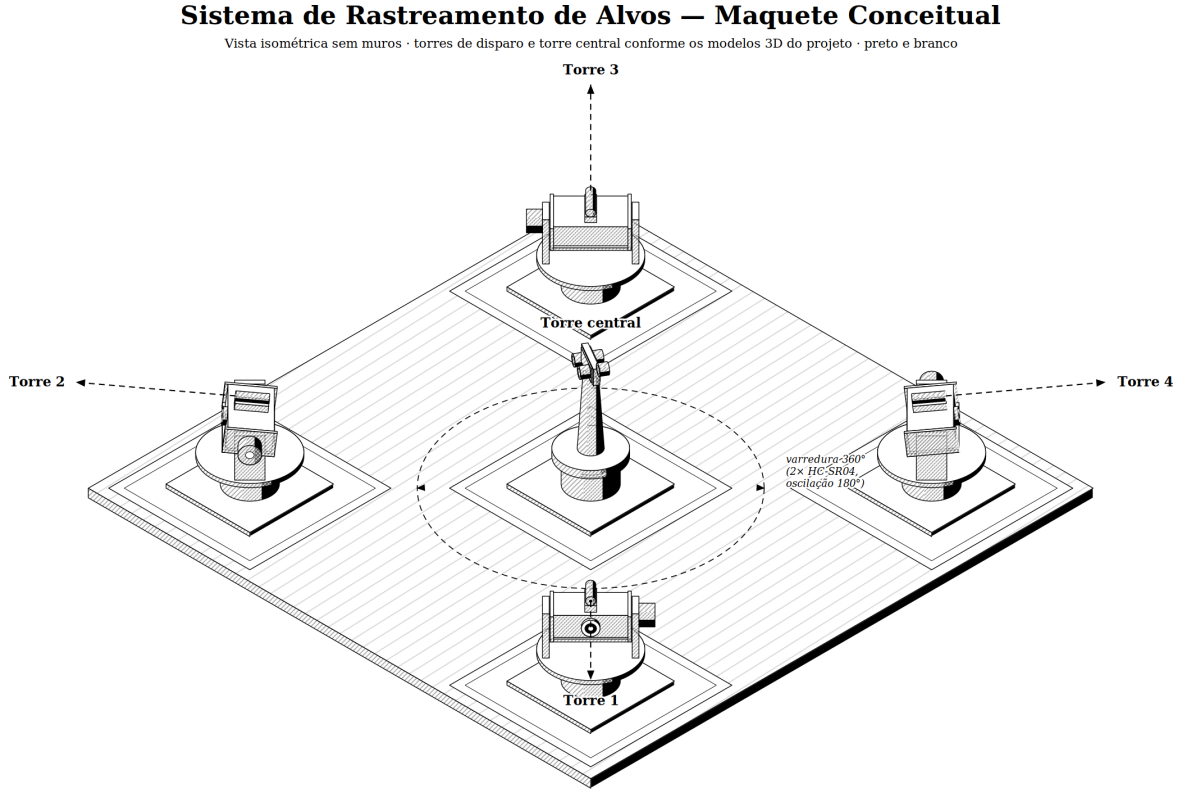


Figure 1: Maquete conceitual do arranjo das cinco torres (vista isométrica): quatro torres de disparo e a torre central com varredura de 360° (dos autores).

2.3.3 Módulo de visão

O módulo de visão utiliza quatro câmeras de 720p conectadas por USB ao computador, associadas aos quadrantes do radar, e processa os quadros de vídeo (*frames*) para classificar o alvo por sua cor, formato e tamanho. Cada quadro capturado é entregue já decodificado à detecção, que executa localmente um modelo YOLO26, na variante nano, em formato ONNX, para propor as regiões candidatas da imagem. Cada candidata é então comparada ao perfil do alvo previamente cadastrado – definido por cor, formato e tamanho, e opcionalmente por fotos de referência recortadas no objeto de interesse. Havendo foto de referência, a comparação combina um histograma de matiz e saturação com uma grade espacial de cor média, usados como assinatura do objeto; na ausência de foto, o critério recai sobre a fração de pixels com a cor cadastrada e sobre a classificação de formato. Quando o detector não propõe nenhuma caixa com a cor do perfil, as regiões candidatas passam a ser propostas por segmentação de cor, como último recurso na ordenação de candidatos.

O envio das informações resultantes ao Arduino, para guiar o apontamento da torre (movimentos de pan e tilt), foi projetado mas ainda não está implementado: o protocolo serial atual não possui comando de apontamento, e a fusão entre a posição estimada pela visão e a distância medida pelos sensores ultrassônicos permanece como etapa planejada. Como o hardware ainda não foi montado, a validação funcional descrita neste trabalho foi feita por simulação, e nenhum resultado de campo é relatado.

PARECER DO PROFESSOR – PROBLEMA GRAVE: Esta afirmação contradiz diretamente o firmware já existente no repositório. O cabeçalho de `RadarTorres_Firmware.ino` documenta, como mensagens NOVAS do protocolo: `CAM;ID=<1-4>;X=<px>;Y=<px>;Z=<cm>` (deslocamento do alvo em relação ao centro da imagem, calculado pela visão do PC, exatamente a informação que o artigo diz que "não tem comando para enviar"), `CALIB;CAM=<1-4>;PASSOS_POR_PX_PAN=...;PASSOS_POR_PX_TILT=...` (calibração pixel→passo) e `AIM;TOWER=<1-4>;ANGLE=...` (apontamento da torre). Ou seja: o comando de apontamento **já existe e já está especificado no protocolo**, mesmo que o hardware físico ainda não tenha sido montado para executá-lo de fato. Isso não é uma contradição pequena – é a diferença entre "projetamos, mas não especificamos o protocolo" (o que o artigo afirma) e "o protocolo já está especificado e implementado no firmware, falta só montar o hardware físico" (o que o repositório mostra). Corrijam a frase para refletir o estado real: protocolo especificado e implementado em firmware, execução física pendente de montagem.

PARECER DO PROFESSOR – AJUSTAR: Mais uma lacuna na mesma seção: o firmware também introduz uma máquina de estados de segurança nova, `COLOR;STATE=VERDE|AMARELO|VERMELHO` (um "semáforo" que só autoriza o comando `FIRE` quando está em VERMELHO), totalmente ausente de "Fluxo operacional" e das regras de segurança descritas no artigo. Se este semáforo é parte do projeto final, ele precisa aparecer no fluxograma e na explicação de regras de segurança – hoje o artigo descreve só a distância mínima e as regras configuráveis do `TowerSelectionService`, sem mencionar esse mecanismo adicional já presente no firmware.

2.3.4 Fluxo operacional

Em operação, o sistema segue estas etapas: (1) o Arduino varre o ambiente com os dois sensores ultrassônicos e envia ângulo e distância à aplicação; (2) a aplicação converte cada leitura em posição cartesiana, determina o quadrante, exibe o alvo no radar e registra a

detecção; (3) em paralelo, as câmeras fornecem quadros de vídeo, e o módulo de visão classifica o alvo por cor, formato e tamanho; (4) a aplicação seleciona, entre as torres armadas disponíveis, a mais adequada para o alvo, priorizando as do mesmo quadrante e, entre elas, a de menor distância euclidiana (por proximidade, alvo a alvo); (5) o comando é enviado ao Arduino, que aciona a torre selecionada; (6) o acionamento demonstrativo (laser de baixa potência ou LED) só ocorre respeitando a distância mínima de segurança e as demais regras de segurança configuráveis do sistema; (7) toda ação executada é registrada em auditoria.

A seleção de torre e o acionamento dependem do modo de operação escolhido pelo operador, em um modelo de três níveis: no primeiro, os alvos são apenas detectados e exibidos; no segundo, acrescenta-se a seleção automática da torre, sem acionamento; no terceiro, acrescenta-se o acionamento demonstrativo automático. Quando a regra de segurança reprova um acionamento, ele é bloqueado e o motivo é registrado no console de eventos.

PARECER DO PROFESSOR – AJUSTAR: Os 7 passos estão claros, mas nenhum vem com frequência – e o firmware já define números exatos: alternância entre os 2 sensores ultrassônicos a cada 60 ms (`INTERVALO_LEITURA_US_MS`, ≈8,3 Hz por sensor) e laço de correção de mira das câmeras a cada 50 ms (`INTERVALO_CONTROLE_CAMERA_MS`, 20 Hz). Uma frase depois do passo (1) – "cada sensor é lido a ≈8 Hz, alternadamente" – custa uma linha e torna a descrição quantitativa, que é o padrão esperado de um artigo de engenharia que usa "tempo real" no resumo.

A BANCA VAI PERGUNTAR: O próprio comentário no firmware diz que 50 ms foi escolhido "ajustando observando o comportamento" – empírico, por tentativa. Por que não 20 ms (50 Hz) ou 100 ms (10 Hz)? Uma frase relacionando a frequência escolhida à velocidade angular máxima esperada do alvo já bastaria para defender o número na banca.

2.3.5 Controlador neuroevolutivo: heurística e rede neural combinadas

O passo (4) do fluxo operacional – escolher, entre as torres armadas disponíveis, a mais adequada para cada alvo, e decidir o instante do acionamento – é resolvido no RadarTorres pela combinação de dois mecanismos de natureza distinta: uma **heurística** determinística, escrita como regras explícitas de engenharia, e uma **rede neural** *feedforward*, cujo comportamento é aprendido por um algoritmo evolutivo próprio, sem biblioteca de aprendizado de máquina. Ambos coexistem no mesmo controlador híbrido, e é essa combinação – não cada mecanismo isoladamente – que determina tanto o comportamento em produção quanto o que é efetivamente treinado.

A heurística é um conjunto de regras fixas, escritas e revisáveis por engenharia, cujo resultado para uma mesma entrada é sempre o mesmo. No RadarTorres ela resolve, em cascata, três camadas de prioridade na seleção da torre: primeiro considera apenas as torres livres do próprio quadrante do alvo; na ausência de candidata, estende-se às torres de quadrante adjacente cujo quadrante de origem não tenha outro alvo concorrente; por fim, na ausência de qualquer uma das duas, recorre a qualquer torre livre restante, de modo que nenhum alvo fique sem cobertura. Cada camada carrega um fator numérico (1,0; 0,5; ou 0,0) que soma-se diretamente à pontuação da candidata. A heurística também impõe cinco regras de segurança e refinamento, cada uma ativável individualmente no painel de Configurações: anti-oscilação, que evita trocar de torre por uma diferença de pontuação dentro de uma margem de empate; priorização por urgência, que permite a um alvo

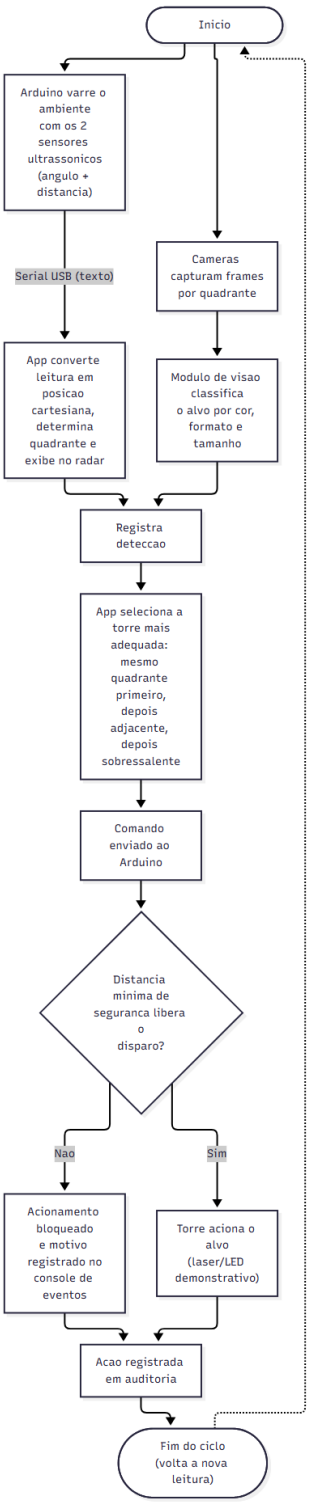


Figure 2: Fluxograma do fluxo operacional do RadarTorres, da leitura dos sensores ao registro em auditoria (dos autores).

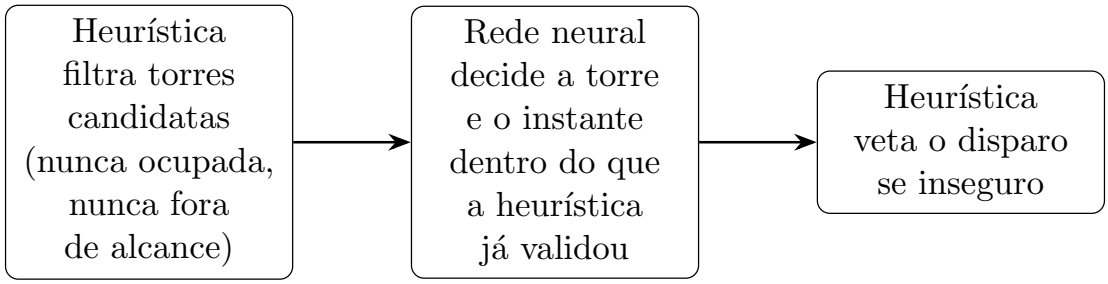
muito próximo da base "tomar" a torre de um alvo mais distante; corredor de tiro, que descarta candidatas cuja linha de disparo atravesse uma zona morta ativa; balanceamento de carga, que prefere, entre torres empatadas, a usada com menor frequência na sessão; e predição de trajetória, que projeta a posição futura do alvo para calcular o ângulo de mira com antecedência de poucos décimos de segundo. Por fim, a heurística impõe dois limites absolutos ao disparo, nunca delegados à rede: um tempo mínimo de mira de 0,3 s e um limiar mínimo de certeza de identificação do alvo (`LimiarConfiancaDisparoPercentual`), abaixo do qual o disparo é recusado independentemente de qualquer outra avaliação.

A rede neural, por sua vez, não decide essas regras de segurança nem a cascata de camadas – ela decide apenas a escolha fina *dentro* do espaço já validado pela heurística. É uma rede *feedforward* totalmente conectada, implementada do zero (sem biblioteca de aprendizado de máquina), com arquitetura de 6 entradas, uma camada oculta de 8 neurônios e 2 saídas, com ativação tanh em todas as camadas. As seis entradas – todas normalizadas no intervalo $[0, 1]$ ou equivalente – são a distância normalizada ao alvo, dois indicadores binários de quadrante (mesmo quadrante e quadrante adjacente), o ângulo de mira normalizado, a velocidade normalizada do alvo e o próprio fator da camada heurística vigente, usado como sinal de contexto. As duas saídas são um ajuste de pontuação, somado à pontuação heurística da candidata, e um valor de confiança de disparo, cujo sinal positivo autoriza o acionamento. Os pesos e vieses da rede – 74 parâmetros, no total, para essa arquitetura – não são ajustados por retropropagação, mas evoluídos por um algoritmo genético: cada genoma é um vetor plano desses 74 números, a população evolui por seleção por torneio, *crossover* uniforme, mutação gaussiana e elitismo, e o genoma final é embutido na aplicação como um vetor de pesos fixo, sem qualquer etapa de treinamento em tempo de execução.

A combinação entre os dois mecanismos é, portanto, sequencial e hierárquica, nunca uma mistura indiferenciada: a heurística primeiro filtra quais torres são elegíveis e calcula as *features* de entrada (quadrante, ângulo, fator de camada); a rede neural, em seguida, pondera essa pontuação dentro do subconjunto já validado, decidindo qual candidata específica vence o empate fino que a heurística, por si só, não resolveria com a mesma sensibilidade; e, por fim, a heurística volta a atuar como filtro de segurança sobre a decisão de disparo, vetando qualquer autorização da rede que viole o tempo mínimo de mira, a certeza mínima ou uma zona morta ativa. Essa hierarquia tem uma consequência deliberada para o pior caso: uma generalização imperfeita da rede nunca produz uma decisão arbitrária, porque toda saída da rede continua restrita ao espaço de candidatas e ao conjunto de disparos já pré-aprovados pela heurística – no limite, o sistema regride ao comportamento puramente heurístico, nunca a um erro fora de controle.

Esse acoplamento se reflete diretamente no treinamento do controlador. O algoritmo genético não avalia a rede neural isolada: avalia o comportamento do `ControladorHibrido` completo – heurística e rede juntas – em episódios simulados de 45 s, com as torres reais do projeto, alvos nascendo na borda de um mundo circular e avançando em direção à base. O fitness de cada genoma soma recompensas por abate (maiores quanto mais longe da base o alvo é interceptado) e penalidades por alvo que escapa ou cruza um ponto de perda sem decisão, de modo que o genoma é pressionado a aprender a operar bem *dentro* das restrições que a heurística já impõe, e não a contorná-las – não há genoma "vencedor" que dispare fora da zona morta ou abaixo da certeza mínima, porque essas decisões nunca chegam a ser avaliadas como tal pelo am-

biente de treino. Essa mesma dependência, no entanto, também é o ponto mais sensível do processo: como o fitness depende inteiramente do comportamento simulado, qualquer premissa equivocada no ambiente de treino se propaga, silenciosamente, para o genoma evoluído. Foi exatamente esse o caso identificado e corrigido neste trabalho – a certeza de identificação do alvo era sorteada uma única vez, no nascimento, e nunca era atualizada, de modo que cerca de 70% dos alvos simulados nasciam permanentemente abaixo do limiar mínimo de disparo e jamais podiam ser abatidos, independentemente da qualidade do genoma, o que mascarava qualquer evolução real sob a aparência de estagnação. A correção – fazer a certeza crescer com o tempo de observação do alvo, como ocorreria com uma câmera real acumulando quadros – e o alinhamento posterior dos parâmetros de alcance de mira e de limiar de certeza usados no treino com os mesmos valores configuráveis em produção (`ConfiguracoesAplicativo.AlcanceMiraMetros`, `LimiarConfiancaDisparoPercentual`) ilustram, em conjunto, por que a fidelidade do ambiente simulado ao comportamento do sistema real é, para um controlador neuroevolutivo, tão determinante para o resultado do treinamento quanto a própria arquitetura da rede ou os hiperparâmetros da busca genética.



`ControladorHibrido` – a mesma classe decide no treino simulado e na operação real

Figure 3: Fluxo de decisão do controlador híbrido: a heurística determinística filtra e, ao final, veta; a rede neural decide apenas dentro do espaço já validado (dos autores).

PARECER DO PROFESSOR – BOM, MAS PODE MELHORAR: Ótimo diagrama – resolve bem a necessidade de visualizar a hierarquia filtra→decide→veta, e a nota abaixo da caixa central (mesma classe no treino e na produção) é um detalhe que reforça exatamente o ponto mais original do artigo. Sugestão pequena de refinamento: adicionar uma seta pontilhada de "veta" de volta para "filtra" rotulada "fitness/penalidade" – hoje o diagrama mostra o fluxo de UMA decisão em produção, mas não deixa visualmente explícito que, no treino, o resultado de "veta" (abate/fuga) é o que alimenta o fitness que evolui a própria rede. Não é obrigatório, mas fecharia visualmente o laço treino↔produção que o texto já descreve bem em prosa.

2.4 Diagrama de pacotes

A organização interna da aplicação desktop foi documentada por meio de um diagrama de pacotes, diagrama estrutural da UML que representa a divisão lógica do sistema em pacotes e as dependências existentes entre eles, sem detalhar atributos ou métodos das classes neles contidas. No RadarTorres, cada pacote corresponde a um *namespace* do projeto `RadarTorres.App`, coincidente com uma pasta do código-fonte, e o diagrama foi obtido diretamente do código da *branch* principal do repositório: cada seta representa ao menos uma diretiva `using RadarTorres.App.*` (ou uma referência `clr-namespace` em

XAML) presente nos arquivos do pacote de origem, de modo que o diagrama descreve a estrutura efetivamente implementada, e não apenas a pretendida.

O fluxo principal de dependências segue a organização do padrão MVVM, de cima para baixo: Views → ViewModels → Services → Repositories → Data → Models. O pacote Models ocupa a posição mais estável do sistema – todos os demais dependem dele e ele não depende de nenhum outro –, o que é desejável para as entidades de domínio, e a separação entre as interfaces de Repositories e a implementação em CSV do pacote Data é o que viabiliza a futura migração para um banco relacional sem alterar ViewModels ou Services. A construção do diagrama a partir do código revelou, também, dependências que formam ciclos entre pacotes: o `NavigationService` importa *namespaces* de Views para instanciar telas (Services → Views, fechando o ciclo com Views → ViewModels → Services), e o `DataSeeder`, responsável pela carga inicial de dados, importa Models, Repositories e Services (Data → Repositories/Services, fechando o ciclo com Repositories/Services → Data). Confirmados em `NavigationService.cs` e `DataSeeder.cs` no código atual, esses dois ciclos continuam presentes, mas a figura acima, regenerada ao nível de pacote e não de classe, não os destaca visualmente – ficam registrados apenas neste parágrafo, em vez de perdidos. Eles não impedem o funcionamento da aplicação, pois

PARECER DO PROFESSOR – AJUSTAR: Vocês mesmos já admitem a limitação (bom, honestidade científica), mas "ficam registrados apenas neste parágrafo, em vez de perdidos" é uma meia-solução – um diagrama se lê em 10 segundos, um parágrafo de prosa sobre dependências cíclicas exige o leitor reconstruir mentalmente o grafo. Como já existia uma versão anterior da figura com os ciclos destacados em vermelho, a correção mais barata aqui é regenerar a figura UMA vez mais com esse destaque, em vez de manter a explicação só em texto.

todos os pacotes pertencem ao mesmo *assembly*, mas aumentam o acoplamento e dificultam testes isolados e futuras separações em projetos distintos.

2.5 Modelo de dados

A persistência do RadarTorres foi igualmente documentada por meio de um levantamento do modelo de dados diretamente a partir do código-fonte já implementado. O protótipo não depende de um servidor de banco de dados: utiliza **SQLite** (`Microsoft.Data.Sqlite`, arquivo único `radartorres.db` na pasta de dados do usuário), com `PRAGMA foreign_keys = ON` garantindo chaves estrangeiras reais – estágio mais avançado do que o modelo em CSV descrito em versões anteriores deste documento (ver nota histórica adiante). O schema atual tem onze tabelas, listadas na Tabela 2: sete cadastrais/de auditoria, já cobertas pelo levantamento em `conteudo_modelo_banco_dados.tex` (pasta `Banco_de_Dados` dos documentos entregáveis), e quatro exclusivas do treinamento neuroevolutivo, que persistem a evolução do controlador híbrido (Seção 2.3.5) entre execuções.

O modelo de dados atual (SQLite, 11 tabelas, chaves estrangeiras reais) é apresentado a seguir em três grupos, por legibilidade: o núcleo de usuários e preferências (Figura 5), o núcleo de detecção, câmeras e auditoria (Figura 6) e as tabelas de treinamento neuroevolutivo, independentes entre si (Figura 7) – diagramas entidade-relacionamento levantados a partir do código-fonte do projeto.

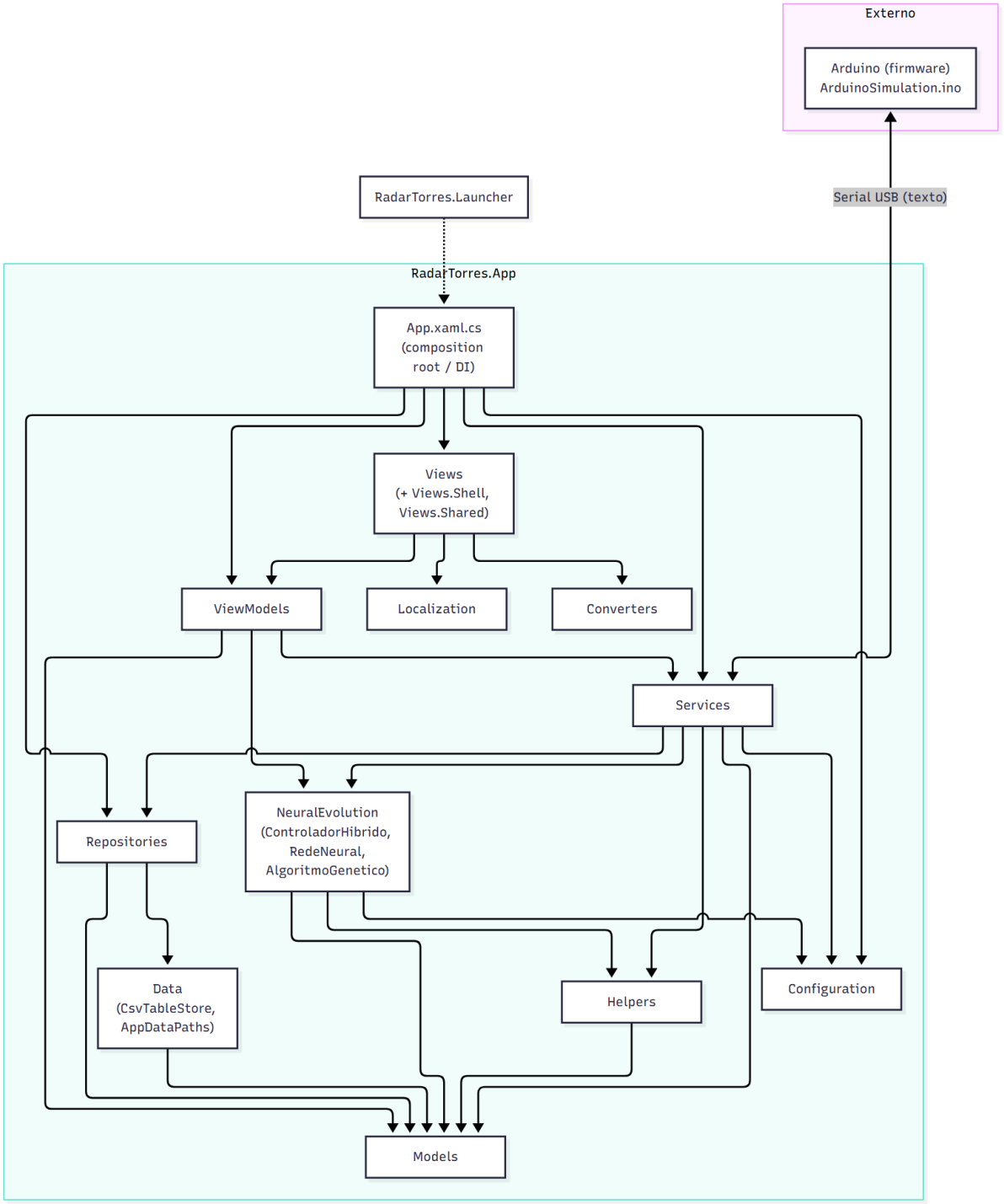


Figure 4: Diagrama de pacotes do RadarTorres. Setas em vermelho indicam dependências que formam ciclos (dos autores).

Table 2: Tabelas do schema SQLite atual (`schema.sql` e os `Sqlite*Repository.cs`).

Tabela	**Finalidade**
`usuarios`	Contas de acesso, perfil (Administrador/Operador/Visualizador) e autenticação
`objetos_detectados`	Histórico de ocorrências de detecção (radar e câmera, por quadrante)
`ocorrencia_estados`	Auditoria das transições de estado de cada ocorrência (só inserção)
`cameras`	Câmera responsável por cada quadrante (Q1–Q4)
`acoes_realizadas`	Auditoria de acionamentos efetivamente executados (só inserção)
`modo_atual_torre`	Auditoria de troca de modo de operação (só inserção)
`preferencias_usuario`	Preferências de interface por usuário (tema, idioma, sidebar)
`classificacoes_alvo`	Classificação de perigo do alvo por velocidade estimada, vinculada a `objetos_detectados` por FK
`historico_treinamento_geracoes`	Fitness (melhor e médio) e contagem de alvos abatidos/escapados a cada geração do algoritmo genético
`marcos_fitness_treinamento`	Agregado de fitness por bloco de gerações, para acompanhar a convergência do treino sem gravar geração a geração
`instantaneos_comportamento_melhor_gene`	Snapshot do comportamento do melhor indivíduo em marcos do treino: fitness, taxa de acerto, taxa de cobertura e o próprio genoma (JSON)

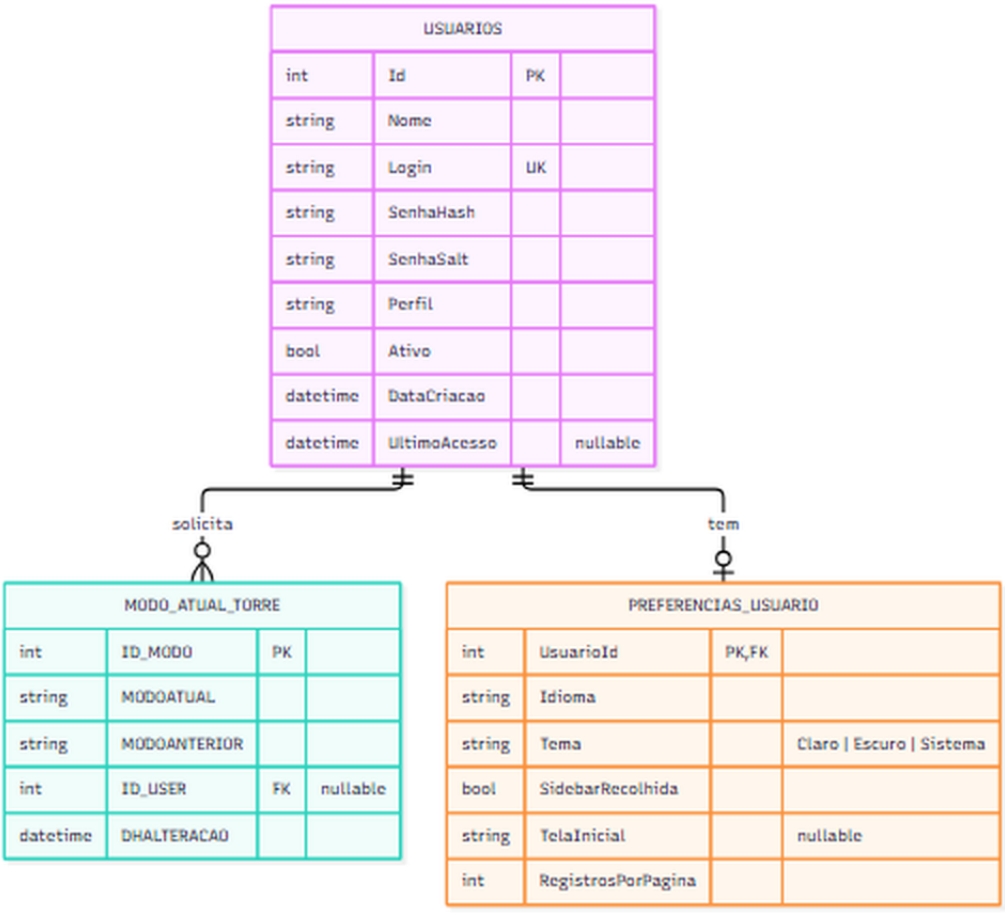


Figure 5: Núcleo de usuários: `usuarios`, `modo_atual_torre` e `preferencias_usuario` (dos autores).

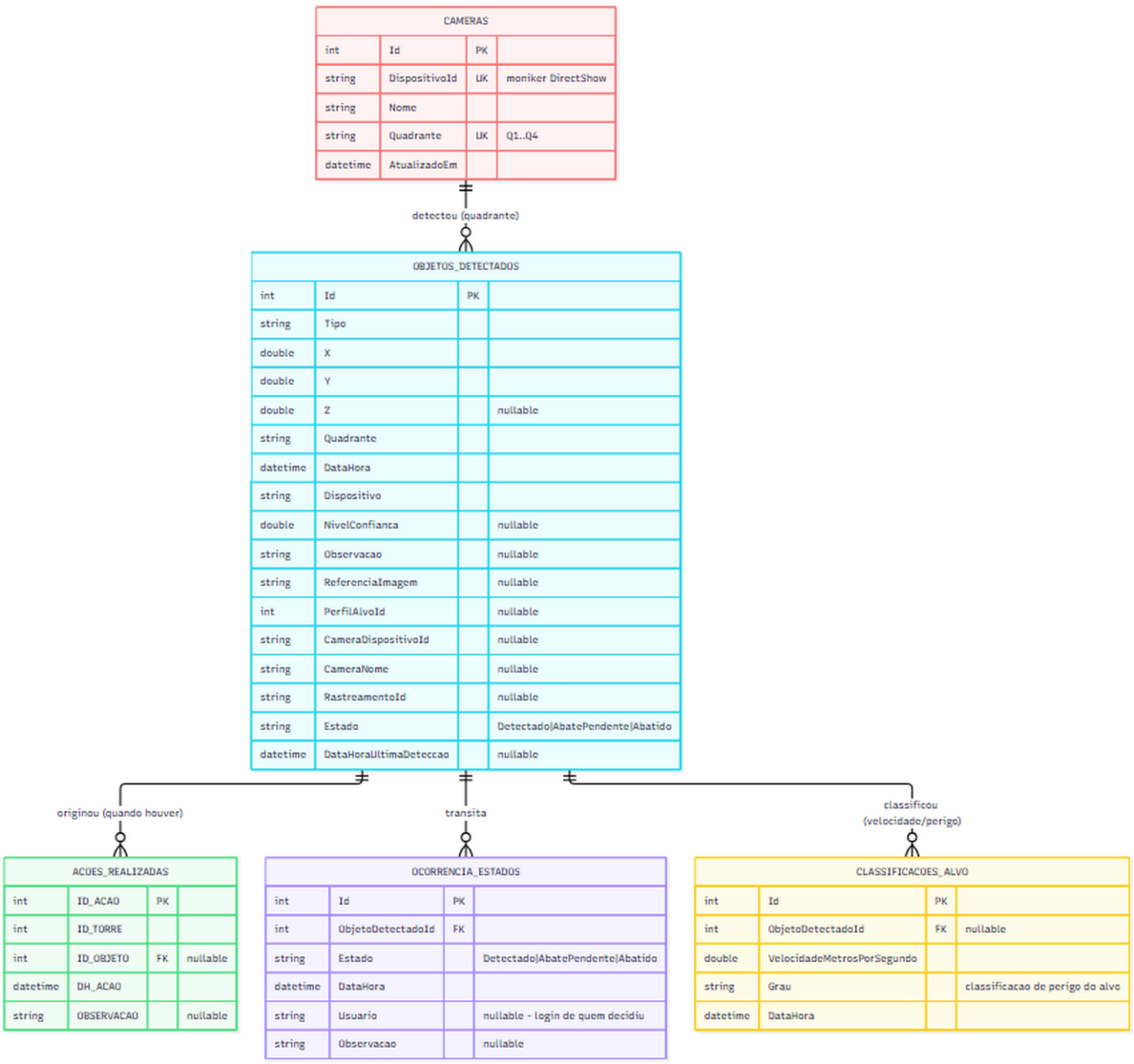


Figure 6: Núcleo de detecção e auditoria: `cameras`, `objetos_detectados`, `acoes_realizadas`, `ocorrencia_estados` e `classificacoes_alvo` (dos autores).

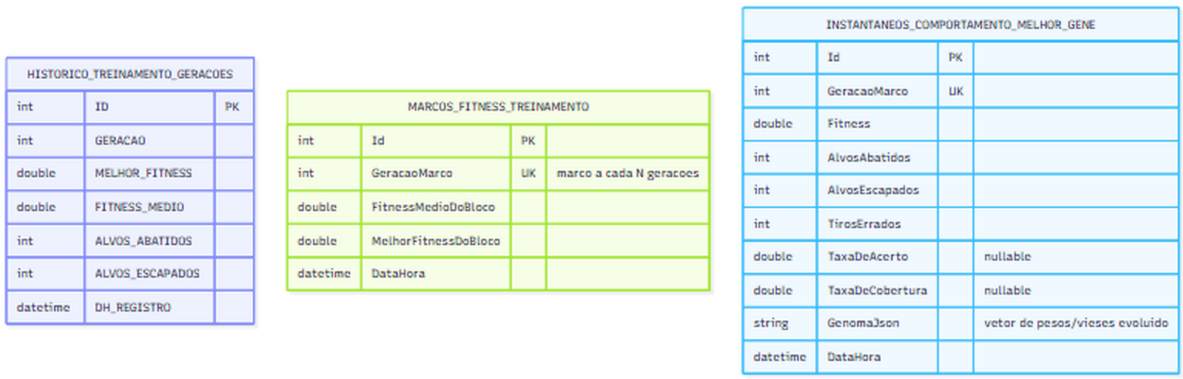


Figure 7: Tabelas de treinamento neuroevolutivo – séries temporais independentes entre si: `historico_treinamento_geracoes`, `marcos_fitness_treinamento` e `instantaneos_comportamento_melhor_gene` (dos autores).

`usuarios` se relaciona 1:1 com `preferencias_usuario` (chave `UsuarioId` → `usuarios.Id`) e 1:N com `modo_atual_torre` (`ID_USER` → `usuarios.Id`) – em ambos os casos por chave estrangeira real de inteiro, não mais por login em texto. `objetos_detectados` se relaciona 1:N com `acoes_realizadas` (quando há alvo associado ao acionamento), com `ocorrencia_estados` (histórico de transição de estado) e com `classificacoes_alvo` (`ObjetoDetectadoId` → `objetos_detectados.Id`); `cameras` se relaciona 1:N com `objetos_detectados` pelo quadrante monitorado. As quatro tabelas de treinamento não se relacionam por FK com as demais nem entre si: são séries temporais independentes, cada uma identificada pelo número da geração (`GERACAO`/`GeracaoMarco`) e pelo instante de registro, cobrindo da granularidade mais fina (`historico_treinamento_geracoes`, uma linha por geração) à mais agregada (`instantaneos_comportamento_melhor_gene`, uma linha por marco, com o genoma completo em JSON).

Como registro histórico do projeto, o modelo original em arquivos CSV tinha seis entidades (sem `ChamadoAjuda`, removida do código) e nenhuma chave estrangeira real – os vínculos entre `Usuario` e `AcaoRealizada`/`ModoAtualTorre` eram feitos por login em texto. A migração efetivamente realizada seguiu a proposta elaborada a partir desse modelo original para as sete entidades que já existiam, mas foi além dela ao acrescentar as quatro tabelas de treinamento neuroevolutivo, inexistentes na época em que a proposta foi desenhada.

3 Conclusão

Este trabalho descreveu o RadarTorres como um protótipo de rastreamento e seleção automática de alvos construído sobre sensoriamento ultrassônico, visão computacional e um controlador híbrido – heurística determinística que filtra e veta, rede neural pequena que decide dentro do espaço já validado, treinada por algoritmo genético. A contribuição mais original deste trabalho é empírica: a identificação e correção, documentada por commit, de uma falha em que a certeza de identificação do alvo era sorteada uma única vez e nunca atualizada, mantendo cerca de 70% dos alvos simulados permanentemente abaixo do limiar de disparo e mascarando a estagnação evolutiva sob a aparência de um treino funcional. A Tabela 1 (Seção anterior) quantifica o efeito dessa correção: o fitness médio da população, estagnado em torno de 9 ao longo de 25.500 gerações antes da correção, passou a acompanhar de perto o fitness do melhor indivíduo depois dela, e a cobertura de alvos disparáveis foi de uma fração minoritária para 100% nas amostras pós-correção.

As limitações do protótipo, discutidas ao longo do texto, concentram-se em três pontos: (i) o hardware físico ainda não foi montado – toda validação é feita em simulação, com modelos (certeza de identificação, probabilidade de erro de disparo) assumidos e não calibrados com dados de campo; (ii) a comunicação entre a visão computacional e o Arduino para guiar o apontamento de pan/tilt foi projetada mas não implementada, permanecendo como integração pendente; e (iii) a propriedade de projeto que limita o controlador híbrido, no pior caso, ao comportamento puramente heurístico (nunca a uma decisão arbitrária da rede) ainda não foi submetida a validação em hardware físico real.

Os trabalhos futuros identificados neste artigo são, portanto: montagem e validação do

hardware físico das cinco torres; implementação do comando de apontamento no protocolo serial e da fusão entre a posição estimada pela visão e a distância ultrassônica; calibração dos modelos de certeza e de erro de disparo com dados de campo, substituindo as premissas simuladas; extensão da seleção de torre para a abordagem multiameaça da literatura de TEWA, hoje deixada de lado em favor da abordagem por proximidade; e, caso a capacidade da rede se mostre o fator limitante do controlador em vez dos hiperparâmetros da busca evolutiva, avaliação de uma arquitetura maior à luz dos resultados de Such et al. (2017).

Parecer final do professor (revisão externa desta versão)

Coerência geral

Esta versão é sensivelmente melhor do que a anterior que revisei: título, resumo e abstract já cobrem o escopo real do trabalho; as referências de TEWA, *action masking* e *shielding* estão completas e bem conectadas à arquitetura híbrida; o diagrama do controlador híbrido é claro; e, principalmente, agora existe uma **Conclusão** com dados quantitativos reais (Tabela 1) sustentando a contribuição mais original do trabalho – a descoberta e correção do bug de certeza estática. Isso é, de fato, nota de TCC bem acima da média: um achado empírico, replicável, com número antes/depois.

O problema que permanece **não é mais** a falta de coerência de escopo (resolvida), mas sim uma **divergência factual entre o que o artigo descreve e o que o próprio repositório já implementa**: a seção de projeto elétrico descreve um Arduino Uno que o firmware físico já documenta como substituído por um Mega 2560, e a seção de visão computacional afirma que não existe comando de apontamento no protocolo quando esse comando já está especificado e implementado no firmware. Some a isso a ausência persistente de uma seção formal de Objetivos, e o quadro é: conteúdo técnico forte, mas desatualizado em dois pontos factuais concretos e estruturalmente incompleto em um ponto (objetivos).

Veredito: APROVADO COM RESSALVAS – menores do que numa revisão anterior, mas ainda bloqueantes para apresentação à banca sem correção. Os itens marcados **PARECER DO PROFESSOR – PROBLEMA GRAVE** nesta revisão (seção de Objetivos ausente, Uno vs. Mega, "não implementado" vs. protocolo já especificado) precisam ser resolvidos antes da entrega final; os marcados **AJUSTAR** são recomendados, mas não impediriam a defesa.

Parâmetros ainda a medir e reportar

Parâmetro	**Onde medir/calcular**	**Estado hoje**	**Objetivo que sustenta**
Qual placa é a final (Uno ou Mega 2560) e qual projeto elétrico vale	Confrontar `Artigo.tex` com `RadarTorres_Firmware.ino`	Divergentes – firmware já documenta a troca	Projeto elétrico coerente e verificável
Estado real do protocolo de apontamento câmera/torre	Mesma comparação, seção de visão	Artigo diz "não implementado"; firmware já tem `CAM/CALIB/AIM`	Comunicação hardware-software documentada corretamente
Hardware/máquina usada no treinamento (CPU, núcleos)	Registrar junto da Tabela 1	Não reportado (só o tempo de relógio, 13h40–14h57)	Reprodutibilidade do experimento de treinamento

Parâmetro	**Onde medir/calcular**	**Estado hoje**	**Objetivo que sustenta**
Ablação de hiperparâmetros (neurônios ocultos, população)	Reexecutar o treino variando 1 parâmetro por vez	Não feito – valores por convenção	Controlador neuroevolutivo com desempenho justificado
Fitness na janela que inclui a geração 5377 (genoma embutido)	Histórico já persistido, `historico_treinamento_geracoes`	Reportada só a janela 1141–1440, não 5377	Rastreabilidade entre experimento reportado e genoma em produção
Frequências do firmware (leitura ultrassônica, laço de mira)	Já definidas no código – `INTERVALO_LEITURA_US_MS`, `INTERVALO_CONTROLE_CAMERA_MS`	Nunca citadas no texto do artigo	Detecção e rastreamento "em tempo real" com número que sustente a afirmação
Baud rate efetivo / latência de round-trip medida	Cronômetro de software, leitura→exibição no radar	9600 bauds padrão, nunca medido nem justificado	Comunicação hardware-software em tempo real
FPS/latência de inferência do YOLO26n por câmera	Benchmark do serviço de detecção, mesmo em máquina de desenvolvimento	Não medido/configurado em nenhum lugar do código	Classificação do alvo em tempo real

Diferente da revisão anterior, a maioria destes parâmetros **já tem a infraestrutura pronta para ser medida** – o histórico de treinamento já existe, o firmware já define as constantes, só falta reportá-las no texto. É, portanto, trabalho de redação e de um benchmark simples de FPS, não de pesquisa nova.

Referências

ALSHIEKH, M. et al. *Safe Reinforcement Learning via Shielding*. 2017.

BAEK, I. et al. *CurbScan: Curb Detection and Tracking Using Multi-Sensor Fusion*. 2020.

HUANG, S.; ONTAÑÓN, S. *A Closer Look at Invalid Action Masking in Policy Gradient Algorithms*. 2020.

MAJONI, L. et al. Camera and ultrasonic sensor fusion for electronic travel aid: design and performance analysis. *MATEC Web of Conferences*, v. 417, p. 10005, 2025.

NAEEM, H. et al. *A Novel Two-Stage Dynamic Decision Support based Optimal Threat Evaluation and Defensive Resource Scheduling Algorithm for Multi Air-borne threats*. 2009.

NAEEM, H. et al. *A Novel Two-Staged Decision Support based Threat Evaluation and Weapon Assignment Algorithm, Asset-based Dynamic Weapon Scheduling using Artificial Intelligence Techniques*. 2009.

SUCH, F. P. et al. *Deep Neuroevolution: Genetic Algorithms Are a Competitive Alternative for Training Deep Neural Networks for Reinforcement Learning*. 2017.